

Trifolio

[cmcleod26/Web-Programming-2026: Repo for spring semester Web Programming project](#)

Names: Connor McLeod, Isabel Tramontano, Cody Ruff

Emails: cruff@students.stonehill.edu, cmcleod@students.stonehill.edu,
itramontano@students.stonehill.edu

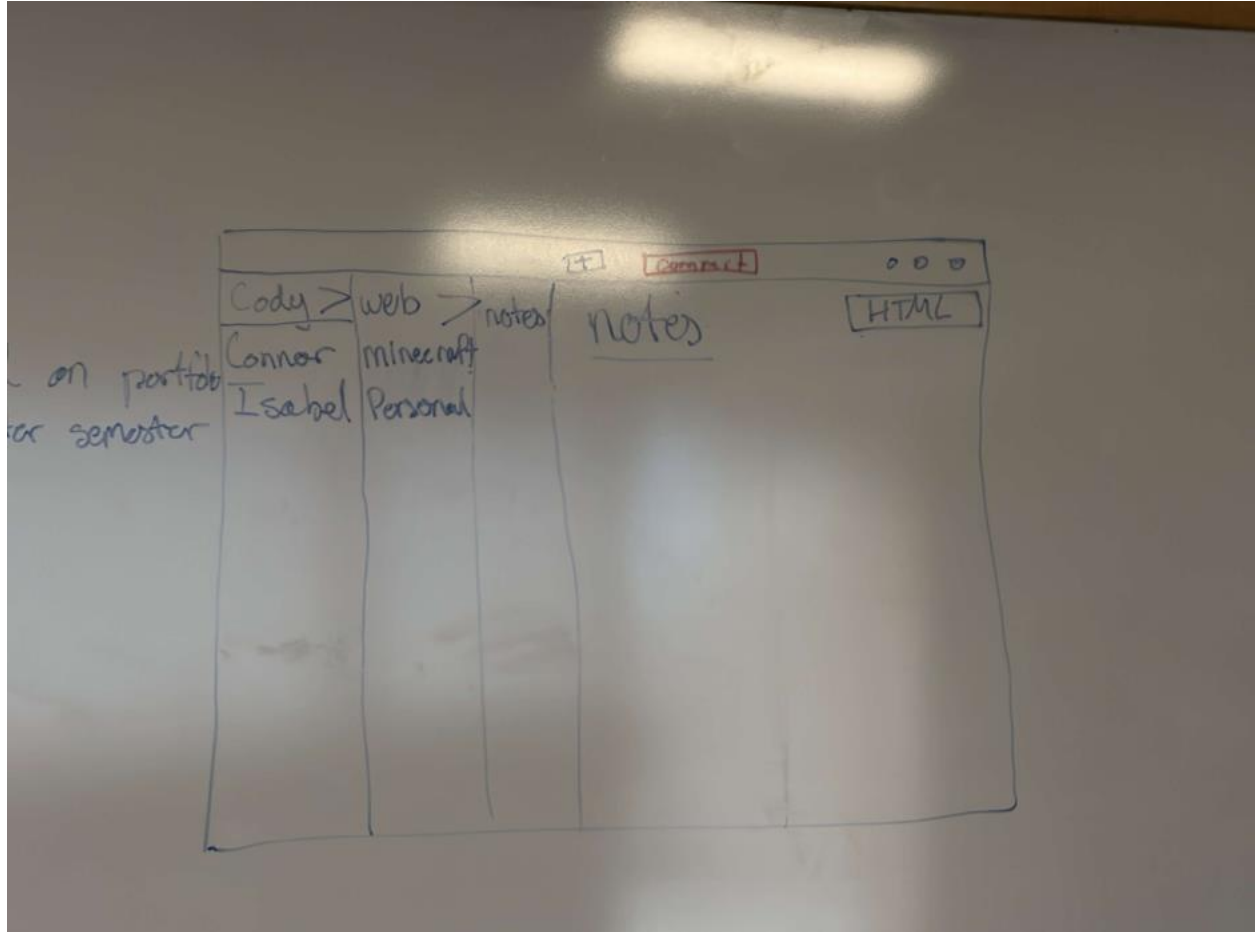
Project Description

We want to build a notes app where users can create folders and store note files inside those folders. There will only be two levels of folders allowed as of right now. This will include a level with a folder for each of the three of us, a layer of personal folders, and then the third layer will contain the notes. Part of the aim is to have a simple, very functional app. When we include a feature, we want to make sure that it is something that we actually want to use and upgrades the overall experience. We also know that this idea has been done thousands of times, so instead of aiming to deploy something for the general masses, this will be an app for the three of us. We do also hopefully want to make these notes computer science oriented and may try to add an element of HTML. One of our approaches to this, will be to have a feature where you can toggle your notes to display it in HTML.

Background and motivation

Our motivation for this is to make a notes app where we can document the creation of said app inside of itself considering the actual practicality of something that is being created for a class or to add to a resume. While this could be something fun to have personally the reality of whatever you build being kept long term is unlikely whether its due to maintenance costs or bug fixes or just life getting in the way this is a way to create something that can be useful without ever needing to be used or updated again as a display of effort and learning.

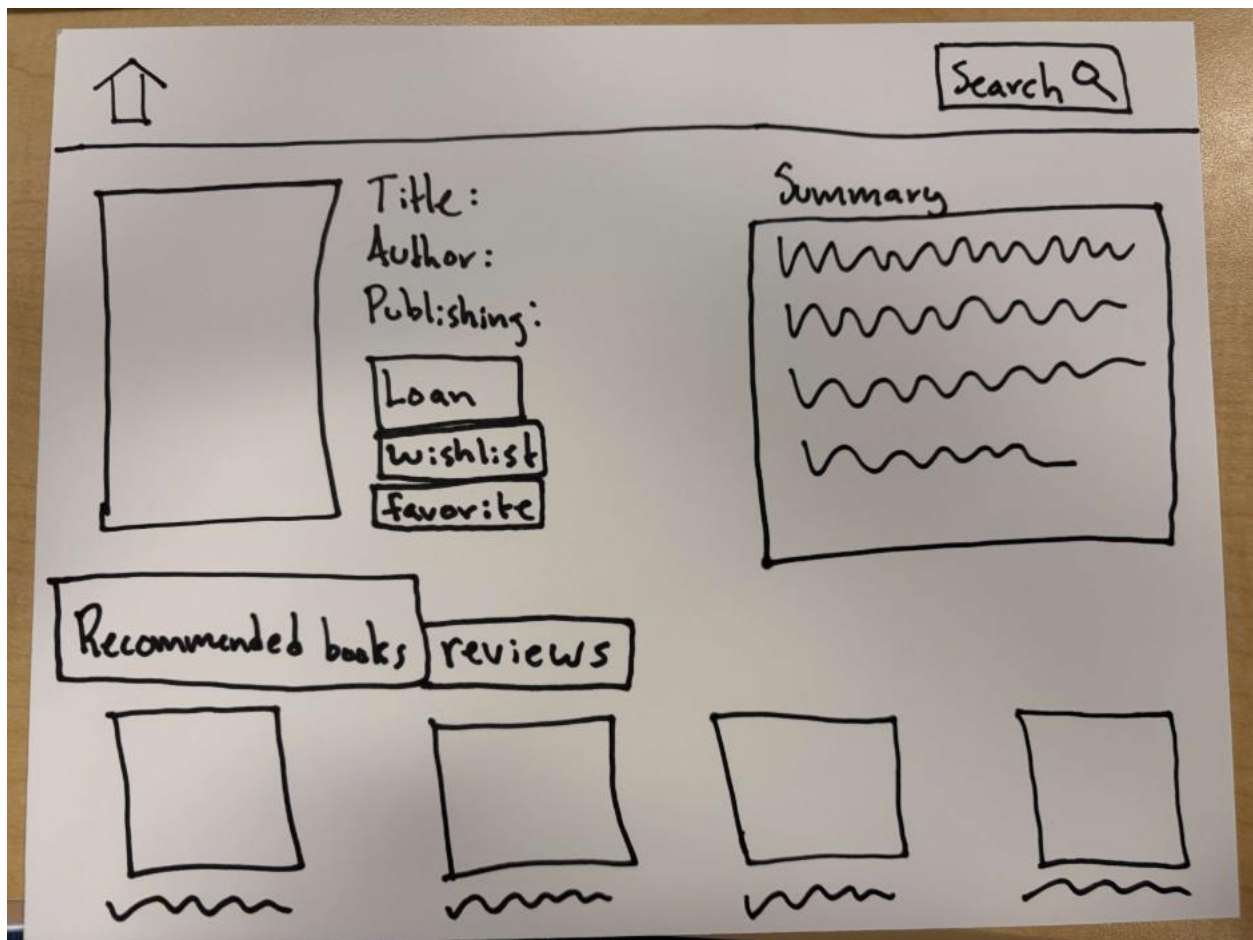
Wireframes and Prototyping

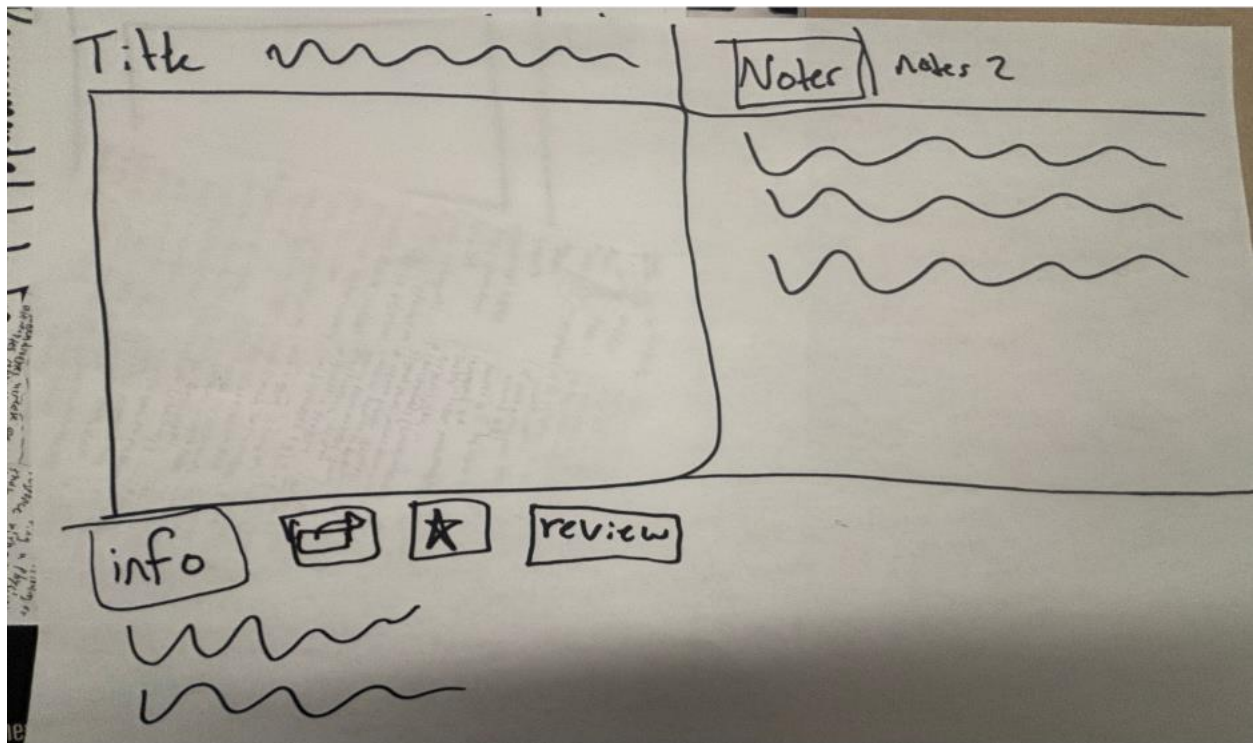


This wire frame should be pretty close to our final product. On the left side, you will see we have our file tree. The first two columns will be folders User>Personal Folders>Notes. On the right side is the window in which we display the note that is currently open. Again I want to reiterate that we are strongly prioritizing fundamental functionality and not drowning ourselves in features. For our other planned features, we have an HTML button in the top right that would allow the viewer to see their notes as HTML code. Then have an upload button which looks like a small button with a plus on it. This will be to upload text files, or to upload PDFs if we add on that functionality. Then we have a commit button which is what we will use to upload our changes. Our reason for including this is we want to make sure that we can control who is making changes while allowing anyone to view our notes. Our other option would be to make users, however we don't want to deal with the logic of permissions. Instead, you will just need to have a password to commit.

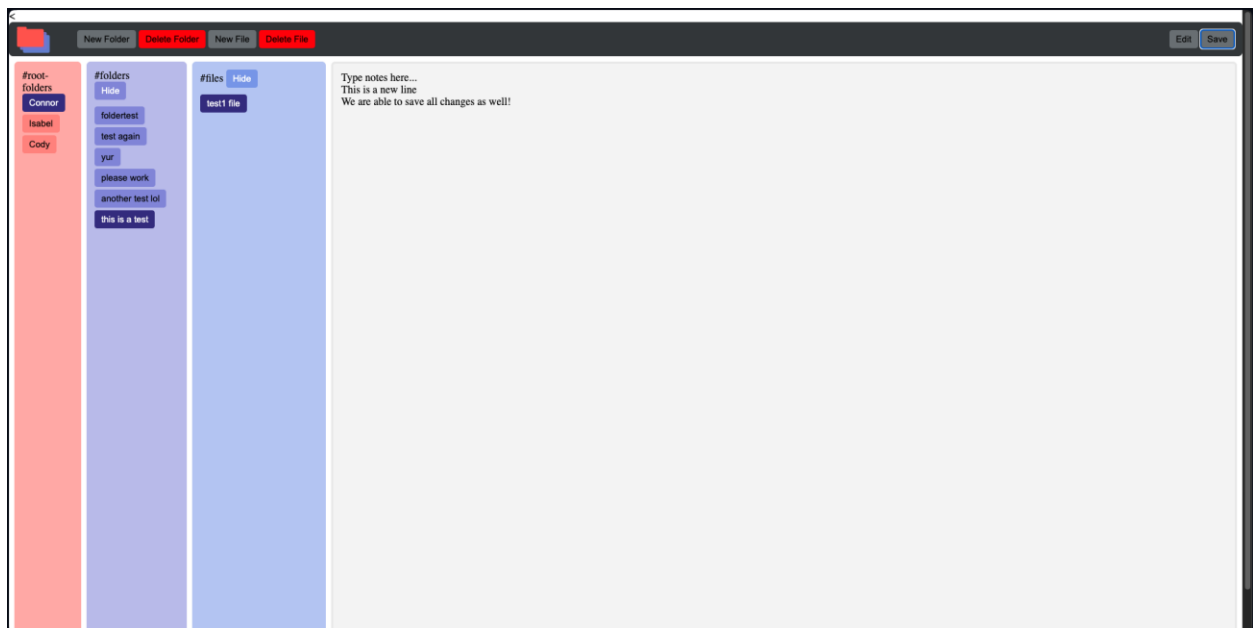
The following wireframe is Isabel's proposal for our project. The main image would be the book image, followed by all the information of the book; title, author, publishing date, and a summary of the book. For the main features, we have the three buttons underneath the book's information. Starting with the Wishlist button, which would add the book to a wish list, and the favorites button adds the book to a favorites tab. The Loan button would allow you to loan out the book, if the book was already taken then the button wouldn't be available. Upon pressing the loan button, it would go to a page like page 2, where you can write notes about the book in the side tab. Finally, we had a recommended and reviews tab, which would let you see books similar to the current one, and the review tab would let you see other people's reviews on the book and make one yourself.

While discussing the proposals, we switched the idea from taking notes on books to taking notes on an uploaded pdf, with a side tab similar to the one on page 2. While we decided to go with the note taking app, we wanted to also include features from her idea. Since both of the ideas had a note taking element, we kept the ability of adding a pdf and creating notes in a side tab.



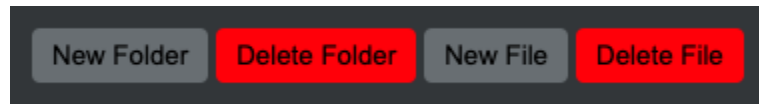


Final Version



Must have Features

- User folders which will only be Connor Cody and Isabel hardcoded.
 - Completely done
- There must be folders able to be created inside these user profiles.
 - Completely done
- There must be the ability to add notes/text files inside the folders.
 - Completely done
- There Must be a delete folder or delete file option
 - Completely done



-
- There must be a password protection to save changes so not anyone accessing the site can update it – Commit button
 - Completely done. There is a password for each of us
- Ability to change files names
 - Not done – we did not have time to complete this and there are ways to achieve this without a change button. Someone can create a new note and just copy & paste into it
- Ability to change files names

Optional

- Pdf uploads to have side by side note taking of pdf files, whether its books or chapters or story descriptions of images.
 - Not done – simply did not have time to get to it
- Text conversion to html style display/ text interpretation of raw text that displays in different ways based off text combination patterns.
 - Not done – simply did not have time to get to it
- Ability to order files and folders
 - Not done – simply did not have time to get to it
- Upload text files
 - Not done – simply did not have time to get to it

Project Schedule

- Creating the interface - connor + Isabel
- Creating the buttons to interact - Cody

- Formatting the database + Connor
- The creation of files/folders + Cody
- The deletion of files/folders Cody + Connor
- The saving of text - Isabel
- The interpretation of text to display it nicely
- Adding functionality to include pdf files in a side by side view with notes

Design Process

Root Folders (Cody)

How it works: The `RootFolder` class in `setupRoots.js` takes a name and manager reference. `CreateRootInHTML()` builds a div with a button inside it, the button has a click listener that calls `manager.setSelectedRoot(this)`. `SetupRoots()` hardcodes the array `['Connor', 'Isabel', 'Cody']` and loops through it creating a `RootFolder` for each one and pushing them into `manager.roots`, its called from the `stateManager` constructor on load.

Differs from wireframe: No, this is exactly what we planned. Three hardcoded root folders one per person.

Challenges: Nothing significant, since theyre hardcoded theres no async stuff to deal with so it was the most straightforward part of the whole frontend.

Folders & Files (Connor)

How it works: The `Folder` class in `Folder.js` stores the name, a files array, a manager reference, and a mongold that starts as null. `CreateFolderInHTML()` creates the button and div and appends to `#folders`, the button calls `manager.setSelectedFolder(this)` when clicked.

On load `loadAllFolders()` in `loadFolders.js` loops through the three owner names and GETs `/api/folders/:owner` for each. For every folder it gets back it creates a folder object, sets mongold, and hides the div right away so you only see the selected roots folders.

When you click New Folder, `createFolderHandler` in `CreateFolders.js` checks a root is selected, prompts for a name and password, and POSTs to `/api/folders`. If we get a 201 back it builds the folder object, calls `createFolderInHTML()`, assigns `data._id` as the mongold and adds it to `manager.folders` and `selectedRoot.folders`. Files work the same

way with the `FileItem` class in `createFile.js` except files are loaded the first time you select a folder if `folder.files.length` is 0.

Differs from wireframe: The wireframe showed folders and files as a list which we matched but we never really thought about what happens when the page loads and everything has to come from a database, that added a whole async layer we didnt plan for.

Challenges: Early on we were creating files before the folders mongold had come back from the server so the `folderId` in the POST body was undefined. The fix was awaiting the POST response before building the local object and assigning `data._id`. The git commit for this particular session says 'after many struggles we can now add files to the database. Holy merge hell.'

State Manager (Cody)

how it works: `stateManager` in `stateManager.js` holds everything together. The constructor sets up all the arrays and null references then calls all the setup functions passing itself so they all share the same state. The three `setSelected` methods all follow the same pattern, remove the selected class from the old button, update the reference, add it to the new button, then update what shows in the DOM. `SetSelectedRoot` hides all folders and shows only the ones belonging to the new root and clears the file view too, `setSelectedFolder` shows only that folders files and triggers a database load if the folder hasnt been opened before, `setSelectedFile` updates the button and enables the edit and save buttons.

Differs from wireframe: This wasnt in the wireframe, it came up during coding because we needed somewhere to track whats selected without every module having to know about every other module.

Challenges: Deciding what state lives on the manager vs on individual objects. Folders have their own files array and the manager also has a flat files array, we kept both because the flat one lets you hide everything in one loop without traversing the whole tree.

Note Pad (Isabel)

How it works: The note pad is the big panel on the right (`#note-pad`). It has two states, read mode where `#note-text` shows content as paragraphs, and edit mode where `#note-editor` shows a textarea. By default it says No File Selected. `SetupNotepad(manager)` in `note.js` grabs all the DOM elements and wires the edit and save buttons.

Differs from wireframe: Yes the wireframe had a single commit button, we changed it to separate edit and save buttons based on the instructors feedback. Its a substantial

change, before it was one button that handled everything and now its a two step process where you enter edit mode then explicitly save.

What led to the change: Instructor feedback. The commit button didnt really make sense as the way to save notes, edit and save is more of what youd expect from a notes app.

Challenges: `note.js` was originally a class and it conflicted with `toggleFolders.js` in a way where if one worked the other broke. Changing it to a regular function fixed the problem.

Editing (Isabel)

How it works: Clicking edit grabs all the `p` elements from `#note-text`, joins their text with `'\n'` as a separator, and puts that into the textarea. Then it hides `#note-text`, shows the textarea, and hides the edit button so you know youre in edit mode.

Differs from wireframe: Yes substantially. The wireframe had no edit state at all it was just commit. Now you enter edit mode before you can type which is a completely different interaction flow.

What led to the change: Instructor recommendation, it also just makes more sense to be able to look at the note and then decide to edit rather than clicking a commit button after everything is already changed.

Challenges: Same class conflict as in the note pad section, switching to a plain function fixed it. Hiding the edit button while editing was a small design decision to stop the user from clicking it again before saving.

Saving (Cody)

How it works: `Save` clears all the `div` elements from `#note-text` then splits the textarea value on newlines and rebuilds a `div` and `p` element for each line, appends them to `#note-text`, then hides the textarea and brings the edit button back.

Differs from wireframe: Yes the wireframe had one commit button that authenticated and saved. Now `save` just updates the DOM and the password only gets asked for on create and delete.

What led to the change: Instructor feedback, separating editing from authentication makes the app feel less clunky.

Backend persistence: There is a `PUT /api/files/:id` route on the server that accepts name, content, and password and updates the file in mongo but the save button doesnt call it yet so notes dont survive a page refresh. That's the next thing to wire up.

Toggling Folders/Files (Isabel)

How it works: The `FolderToggle` class in `toggleFolders.js` takes a `divId`, `hideButtonId`, and `showButtonId`. `Hide()` sets `display none` on the panel and if its the folders panel it also hides files since showing files without folders doesnt make sense. `Show()` brings it back. After each change `updateButtons()` shows or hides the show buttons based on whats currently hidden and `updateLayout()` adjusts the column widths so the remaining columns expand to fill the space.

`SetupFolderToggles()` creates two `FolderToggle` instances one for folders and one for files and calls `setup()` on each. Its imported and called from a script tag in `index.html` separate from `StateManager`.

Differs from wireframe: This wasnt in the wireframe at all, I added it because I thought it would be useful especially if we ever add the pdf side by side view since you would want to maximize the notepad area.

Challenges: Setting `display none` on a flex child removes it from the layout but the other columns dont automatically expand to fill the space. I added `updateLayout()` to manually adjust the widths with javascript depending on whats currently hidden.

Delete Folder / Delete File (Connor + Cody)

how it works: `deleteFolderHandler` in `DeleteFolders.js` checks a folder is selected, prompts for the password, and sends `DELETE /api/folders/:mongold` with the password in the body. On success it loops through `manager.files`, removes any file divs that belonged to that folder from the DOM and rebuilds the array without them, then removes the folder div and removes the folder from `manager.folders` and `selectedRoot.folders`, then clears `selectedFolder`.

On the server `DELETE /api/folders/:id` validates the password then deletes the folder document and runs `File.deleteMany` with the `folderId` so any files inside it get removed too and nothing is left orphaned in the database.

Delete file is the same idea in `DeleteFiles.js`, prompts for password, sends `DELETE /api/files/:mongold`, removes `fileDiv` from the DOM and removes the file from `manager.files` and `selectedFolder.files`.

Differs from wireframe: Delete wasnt in the original wireframe. It was a required feature that just wasnt finished by the milestone.

Was the change substantial: Yes, getting it working meant keeping three things in sync at the same time. The DOM, the manager arrays, and the database. If any one of those gets out of sync the app breaks.

Challenges: Files are tracked in two places, manager.files and the folders own files array, both needed to be cleaned up on delete. Building filtered replacement arrays instead of splicing mid-loop was cleaner and avoided index shifting bugs.

Password Protection (Connor + Cody)

How it works: every write operation uses `window.prompt()` to ask for the password before the request is sent, the password goes into the request body as JSON. On the server each route reads `req.body.password` and compares it to `process.env.COMMIT_PASSWORD`, if it doesn't match it returns 401 and stops. The password lives in a `.env` file that's gitignored so it never gets committed, all three of us have our own `.env` with the same value.

Differs from wireframe: Yes. The wireframe had one commit button that did authentication and saving at the same time. Now every individual write operation requires the password which is a more substantial change than it looks because it means the data is actually protected at the API level not just in the UI.

What led to the change: Instructor feedback. We ended up with one shared password instead of three separate ones because doing real per user auth would need sessions and login pages which is out of scope for this project.

Does it solve a problem; Yes. Before the check was just a frontend thing so anyone who knew the API url could write to the database. Now the check is server side on every route so even using Postman or curl won't let you write without the password.

Backend (Connor)

What is built and working

- Node.js/Express server running on port 3000
- MongoDB Atlas connected as the live database
- REST API with routes
- Password protection on all write operations; `.env` is configured on all three dev environments

- We technically have cors but I don't think it is actually necessary. We honestly haven't removed it though because we don't want to break anything. It's kinda like the TF2 coconut picture

API Routes (Connor + Cody)

folder Routes:

- GET /api/folders/:owner. Finds all folders for a given owner. Called by loadAllFolders() on startup for each user. Returns array of folder docs including _id.
- POST /api/folders. Creates a new folder. Expects { name, owner, password }. Returns 201 and the saved doc on success, 401 on wrong password.
- DELETE /api/folders/:id. Deletes the folder and cascade deletes its files with File.deleteMany. Expects { password }. Returns 401, 404, or 201.

File Routes;

- GET /api/files/:folderId. Finds all files for a given folder. Called by loadFilesForFolder() the first time a folder is selected.
- POST /api/files. Creates a new file. Expects { name, folderid, content, password }. Returns 201 and the saved doc.
- PUT /api/files/:id. Updates name and content. Uses File.findByIdAndUpdate with \$set. Built and tested on the server but not yet called by the frontend Save button.
- DELETE /api/files/:id. Deletes a file by _id. Expects { password }. Returns 401, 404, or 200.

Environment & Config (Connor)

the .env file at the project root needs three things to run the app:

- MONGO_URI. MongoDB Atlas connection string with credentials. Never hardcoded anywhere in the source.
- COMMIT_PASSWORD. The shared password checked on every write route.
- PORT (optional). Defaults to 3000 if not set.

The .env is in .gitignore and has never been committed. All three devs made their own locally with the same values. To run the app: make sure Node.js is installed, run npm install, create the .env, then run node server/server.js and open http://localhost:3000.